



## Module 3 Additional Readings

AI Assisted Coding

### How You Should Use This Resource

- Read the Core readings after the lecture. These are the minimum supporting readings.
- Use the Practical references while building and debugging the Task Tracker Kanban board. They are lookup references, not assigned essays.
- Use Optional deeper dives only after the core work is done. They are not required for the quiz, deliverables, or grading.

### Core Readings: Students Should Read These

Recommended time: about 50-70 minutes total if students focus only on the listed sections. These readings directly support the Module 3 slides, demos, and deliverables.

#### Core 1. GitHub Docs - Best practices for using GitHub Copilot

**Link:** [GitHub Docs - Best practices for using GitHub Copilot](#)

**Purpose:** Use this as the main AI-assisted coding mindset reading for Module 3. It supports the lecture loop: ask, inspect, run, test, and refine.

**Read / use this part:** Focus on creating thoughtful prompts, breaking down complex tasks, providing examples, checking Copilot's work, and using tests/tooling to validate suggestions.

#### Core 2. GitHub Docs - Asking GitHub Copilot questions in your IDE

**Link:** [GitHub Docs - Asking GitHub Copilot questions in your IDE](#)

**Purpose:** Supports Part 3.1: open the real Task Tracker repo, ask Copilot questions from inside the IDE, and verify that it is reading the right code.

**Read / use this part:** Skim the sections on IDE chat, smart actions, and adding context. Pay attention to workflows where you ask about selected code or files instead of asking vague repo-wide questions.

### Core 3. VS Code Docs - Inline chat

**Link:** [VS Code Docs - Inline chat](#)

**Purpose:** Supports Parts 3.3 and 3.4: use inline chat on a selected handler, modal section, render function, or refactor target instead of asking for a full-file rewrite.

**Read / use this part:** Read the editor inline chat section, especially selecting a block of code to scope the prompt and reviewing the inline diff with Keep/Undo.

### Core 4. MDN - HTML Drag and Drop API

**Link:** [MDN - HTML Drag and Drop API](#)

**Purpose:** Supports Part 3.2: the Kanban board uses native browser drag-and-drop, not a drag library.

**Read / use this part:** Focus on the concepts and basic steps: draggable item, transferred data, drop target, dragstart, dragover, and drop. Skip file-upload examples.

### Core 5. MDN - Using the Fetch API

**Link:** [MDN - Using the Fetch API](#)

**Purpose:** Supports fetchTasks, renderBoard, POST/PATCH form submission, and error-state handling in the frontend.

**Read / use this part:** Focus on sending JSON, reading JSON responses, and checking response status. The most important detail for Module 3: fetch does not reject just because the server returns 404 or 422; your code must check status or ok.

### Core 6. GitHub Docs - Writing tests with GitHub Copilot

**Link:** [GitHub Docs - Writing tests with GitHub Copilot](#)

**Purpose:** Supports Part 3.5: use Copilot to brainstorm and draft tests, then verify the tests yourself.

**Read / use this part:** Read the introduction and the Copilot Chat testing workflow. Notice the need for specific test requirements and verification after generation.

## Practical References: Use These While Working

These are not required sit-and-read assignments. Keep them open while completing the hands-on work, checking generated code, diagnosing browser/API issues, or writing tests.

## Practical 1. FastAPI - Testing

**Link:** [FastAPI - Testing](#)

**Purpose:** Use while adding backend tests for PATCH edge cases and status-transition behavior.

**Read / use this part:** Focus on TestClient, naming test functions with test\_, and using the client like httpx. Skip advanced deployment or async topics not used in Module 3.

## Practical 2. pytest - Documentation

**Link:** [pytest - Documentation](#)

**Purpose:** Use as the reference for running pytest, reading failures, and understanding small readable test functions.

**Read / use this part:** Use it as a lookup reference. For Module 3, you mainly need naming conventions, running selected tests, and interpreting failure output.

**Student note:** Do not drift into fixtures, plugins, or large testing architecture unless your instructor assigns it later.

## Practical 3. MDN - DataTransfer

**Link:** [MDN - DataTransfer](#)

**Purpose:** Use while checking dragstart/drop code that stores the task id with dataTransfer and sets move behavior.

**Read / use this part:** Focus on setData/getData, effectAllowed, and dropEffect. Skip clipboard and file-transfer sections.

**Student note:** This is more specific than the main Drag and Drop overview and is useful when Copilot generates confusing dataTransfer logic.

## Practical 4. VS Code Docs - Manage context for AI

**Link:** [VS Code Docs - Manage context for AI](#)

**Purpose:** Use when Copilot answers from generic patterns instead of the actual Task Tracker files.

**Read / use this part:** Focus on adding files, folders, symbols, terminal output, and codebase context to chat. Use this to make prompts grounded in the files students are actually editing.

## Optional Deeper Dives: For Students Who Want More Detail

These readings are useful, but they go beyond what students need to complete Module 3. Use them selectively after the core and practical references.

### Optional 1. GitHub Docs - Prompt engineering for GitHub Copilot Chat

**Link:** [GitHub Docs - Prompt engineering for GitHub Copilot Chat](#)

**Purpose:** Use for extra practice writing prompts that include context, constraints, examples, and output expectations.

**Read / use this part:** Read selectively. The core Module 3 prompts already model the pattern; this is for students who want more general Copilot prompting guidance.

### Optional 2. Martin Fowler / Thoughtworks - Exploring Generative AI

**Link:** [Martin Fowler - Exploring Generative AI](#)

**Purpose:** Use for broader practitioner context on AI-assisted software delivery after students finish the module.

**Read / use this part:** Read selectively. Choose posts about human oversight, AI-assisted delivery practice, or code review. Do not treat the whole series as assigned reading.

**Student note:** This is intentionally optional because Module 3 is a hands-on AI Assisted Coding module, not a general software engineering theory module.